Function in C

Objectives

- Explain User Defined Functions
- Plain Type of Functions
- Discuss category of Functions
- Declaration & Prototypes
- Storage class in "C"
- Calling the Function

Functions

A function is a block of code which only runs when it is called.

Functions are used to perform certain actions, and they are important for reusing code

Two types of function in c :

- Library or Built-In Function
- User-Defined Functions

Library or Built-In Function

- A library function are predefined in the library of the C and also known as a “built-in function”.
- These built function are used to perform the most common operations like calculations, updation, etc.
- To use these functions in the program the user has to use a header file associated with the corresponding function in the program.
- Built-in functions have the advantage of being directly usable without being defined.
- Example
  - `scanf(), printf(),getc(),putc(), pow(), sqrt(), strcmp(), strcpy()`
- Need to include `#include<stdio.h>` preprocessor directive before used.

User-Defined Functions

- A user-defined function is a type of function in C language that is defined by the user himself to perform some specific task.
- It provides code reusability and modularity to your program.
- User-defined functions must be declared and defined before being used.
- Their working is specified by the user and no header file is required for their usage.
- These functions are designed by the user when they are writing any program because not all task have a library functions.

Advantages of User-Defined Functions

- Changeable functions can be modified as per need.
- The Code of these functions is reusable in other programs.
- These functions are easy to understand, debug and maintain.

User Defined Function

Syntaxs

```
return_datatype func_name (arguments) {
        Body of the function
        statements;
        return;
}
```

Call the function from main():

Syntaxs

```
func_name(arguments ) ;
```

```
#include <stdio.h> main(){
        .
        .
      address();
        .
        .
}

void address()
{
        .
        .
        .
}
```

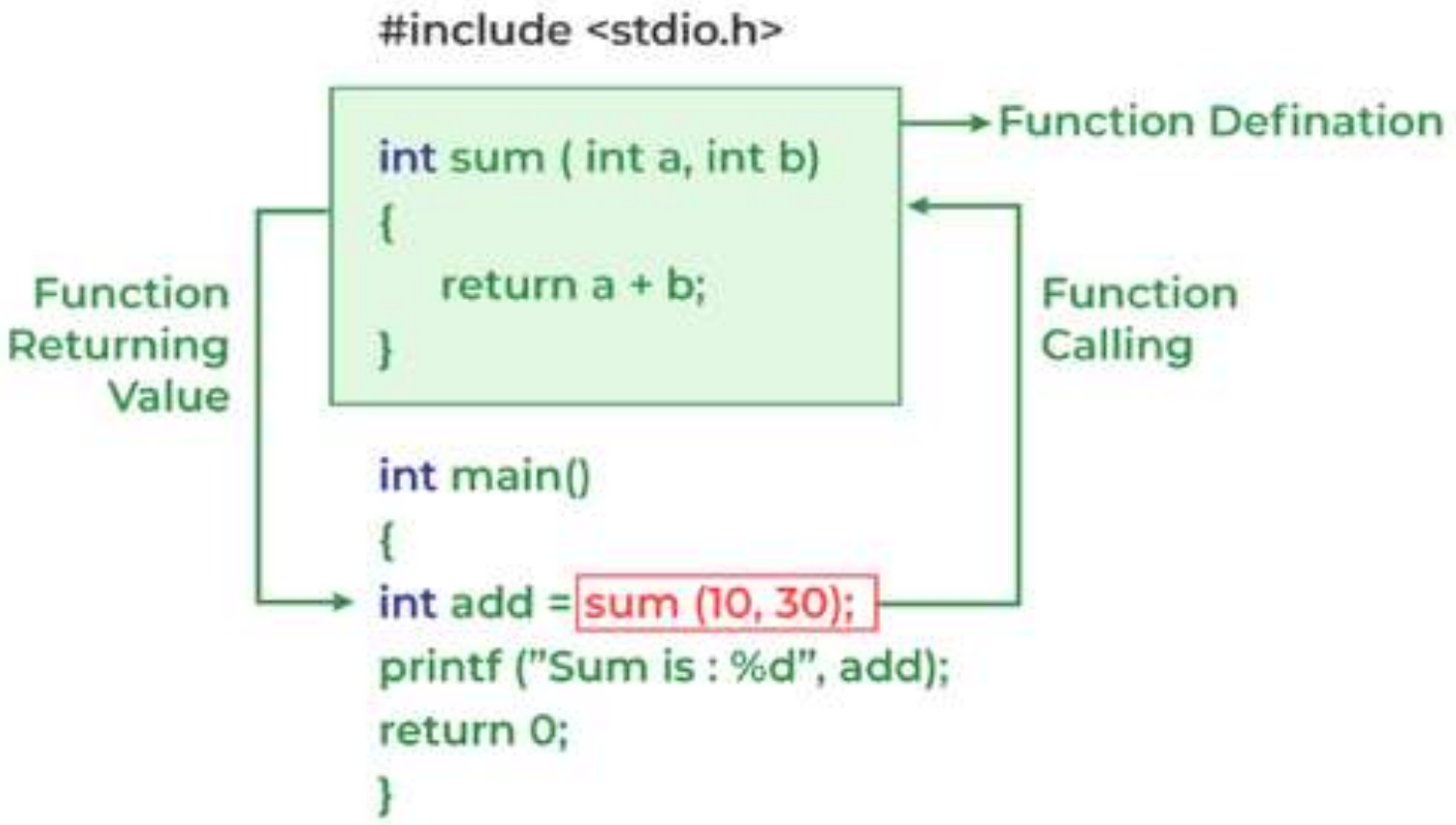
Example 1

```c
#include <stdio.h>

// Function prototype
int sum(int, int);

// Function definition
int sum(int x, int y)
{
        int sum;
        sum = x + y;
        return x + y;
}

int main()
{
        int x = 10, y = 11;
        // Function call
        int result = sum(x, y);
        printf("Sum of %d and %d = %d ", x, y, result);
        return 0;
}
```

Example 2

```
#include <stdio.h>
void test() //definition
{
      printf(" Enter your PIN number");
      printf(" ----------------------");
}

void main()
{
      clrscr();
      printf("Welcome to MyBank \n");
      test(); //calling
      printf("Bye");
      getch();
}
```

Working of Function in C



Category of Function

- Based on return values and passing arguments

1. No argument no return values
2. Argument but no return values
3. No argument with return values
4. With argument with return values

Argument or also known as parameter

Formal and actual parameter

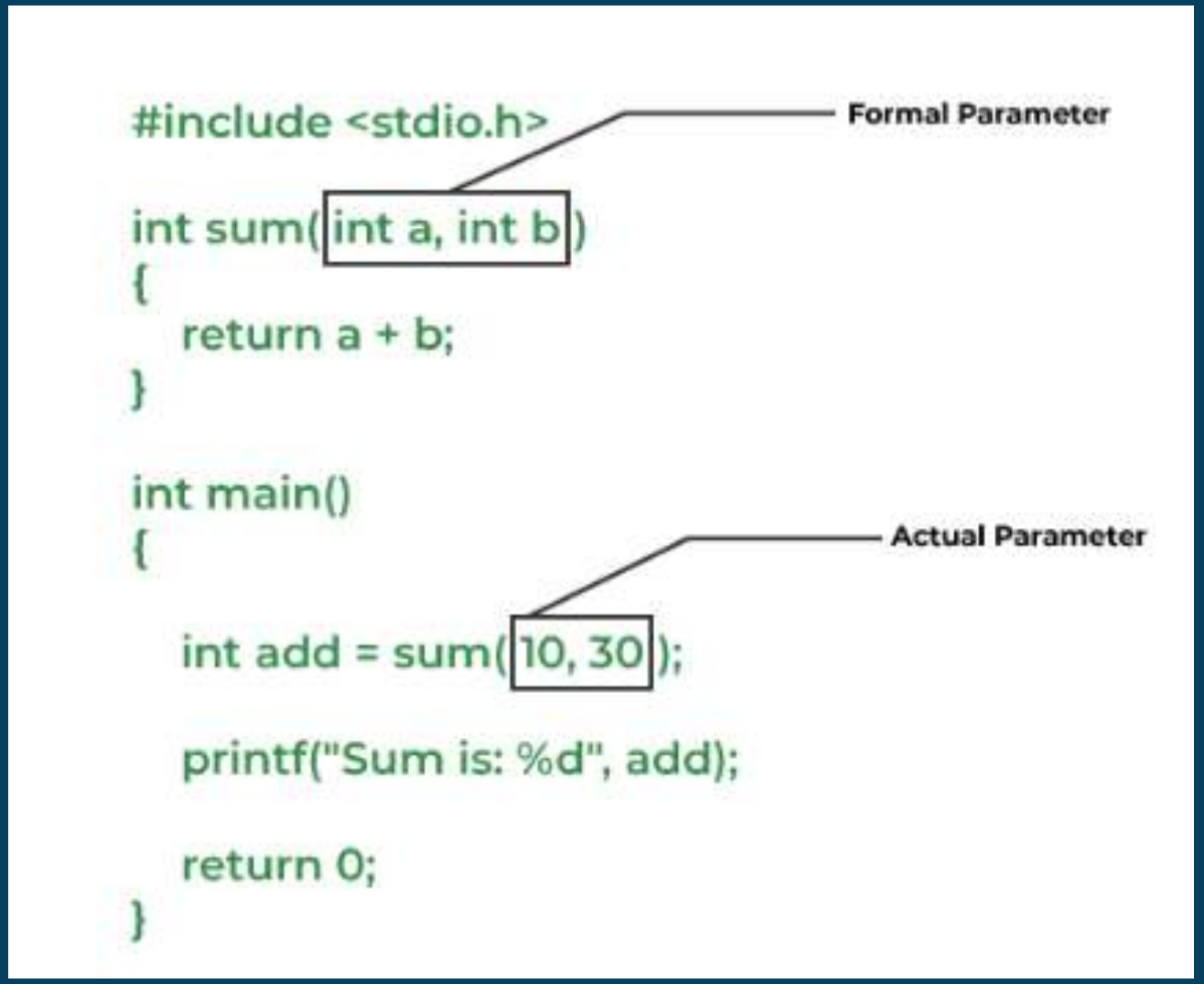
Formal parameter:

The variables declared in the function is known as Formal arguments or parameters.

Actual parameter:

The values that are passed in the function are known as actual arguments or parameters.

Formal and actual parameter



NO ARGUMENT NO RETURN VALUES

```
/* Addition of two numbers */
// NO ARGUMENT NO RETURN VALUES
#include <stdio.h>
void add();
void main()
{
   add();
   printf("main ends here");
   add();
}
```

```
void add() //function without argument
{
   int a, b, c;
   printf("Enter two numbers \n");
   scanf(%d %d, &a, &b);
   c = a + b;
   printf("The sum is %d \n",c);
}
```

WITH ARGUMENT NO RETURN VALUES

```
#include <stdio.h>
void add(int, int);
void main()
{
    int x,y;
    printf("Enter two numbers");
    scanf ("\t %d %d", &x,&y);
    add(x,y);}
```

```
void add(int a, int b) //Formal Argument
{
   int c = a + b;
   printf("\t\tThe c value is %d",c);
}
```

The `return` statement

- The `return` statement is used to return from a function.
- It causes execution to return to the point at which the call to the function was made.
- The `return` statement can have a value with it, which it returns to the program.

WITHOUT ARGUMENT AND WITH RETURN VALUES

```
/* To perform Addition of two numbers without argument and with return values */

#include <stdio.h>

#include <conio.h>

int add(); //declaration

void main()

{

   int c;

   c = add(); // return variable - c

   printf("The sum of two numbers is %d",c);

}
```

```
int add()    // no argument
{
    int a,b,c;
    printf("Enter two numbers = ");
    scanf("%d %d", &a, &b);
    c = a + b;
    return(c);
}
```

WITH ARGUMENT AND WITH RETURN VALUES

```
/* To perform Addition of two numbers With
Argument and with return values */
#include <stdio.h>
int add(int, int);
void main()
{
  int c;
  printf("Enter two numbers = ");
  scanf("%d %d", &a,&b);
  c = add(a,b); // actual argument
  printf("The sum of two numbers is %d",c);
}
```

```
int add(int x, int y)
{
    int c;
    c=x+y;
    return(c);
}
```

Local and Global Variables in C

- Local Variable
  - Local variables are declared within a specific block of code, such as a function or a loop.
  - These variables are only accessible within that block and are typically used for temporary storage or calculations within a limited context.
  - Once the block of code in which a local variable is defined exits, the variable goes out of scope, and its memory is released.
- Global Variables
  - Global variables are declared outside of any function or block of code and can be accessed from any part of the program.
  - Unlike local variables, which have limited scope, global variables have a broader scope and can be used across multiple functions, modules, or files within a program

Example

Local Variable

```c
#include <stdio.h>

void exampleFunction() {
  // Local variable declaration
  int x = 10;
  int y = 20;
  int z = x + y;
  printf("The sum is: %d\n", z);
}

int main() {
  exampleFunction();
  return 0;
}
```

```
The sum is: 30
```

Global Variable

```c
#include <stdio.h>

// Global variable declaration
int global_var = 100;

void exampleFunction() {
  // Local variable declaration
  int x = 10;
  int y = 20;
  int z = x + y + global_var;
  printf("The sum is: %d\n", z);
}

int main() {
  exampleFunction();
  return 0;
}
```

```
The sum is: 130
```

Example : Local variable

```c
#include <stdio.h>

void myFunction() {
  // Local variable that belongs to myFunction
  int x = 5;
}

int main() {
  myFunction();

  // Print the variable x in the main function
  printf("%d", x);
  return 0;
}
```

This code cause an error because variable `x` can't be used outside myFunction function

```
prog.c: In function ‘myFunction’:
prog.c:5:7: warning: unused variable ‘x’ [-Wunused-variable]
    5 |   int x = 5;
      |       ^
prog.c: In function ‘main’:
prog.c:12:16: error: ‘x’ undeclared (first use in this function)
   12 |   printf("%d", x);
      |                ^
prog.c:12:16: note: each undeclared identifier is reported only onc
```

Example : Global variable

```c
#include <stdio.h>

// Global variable x
int x = 5;

void myFunction() {
  // We can use x here
  printf("%d\n", x);
}

int main() {
  myFunction();

  // We can also use x here
  printf("%d\n", x);
  return 0;
}
```

Variable `x` can be used anywhere in the code.

```
5
5
```

Exercises

- Write a program to sort the numbers in ascending order using functions.
- Write a program to calculate x^n using functions
- Write a program to check whether the year is leap year or not using functions
- Write a program to find the square of first N Numbers and to calculate its sum
- Write a program to swap two numbers using functions

Thank you

Baharudin Osman